

CSE 232, Homework 3

Name:

Student ID:

1. (20pts – 5ea) State whether each of the following statements are true or false. Additionally, briefly explain your reasoning.

(a) Leader driven consensus solves uniform consensus in a partially synchronous network.

[Answer: **True.** By definition.]

(b) Every total order broadcast protocol is also a causal order broadcast protocol.

[Answer: **False.** A total order does not necessarily guarantee a causal order. For example, suppose p_1 broadcasts m_1 then m_2 . FIFO order says that every process should deliver m_1 before m_2 everywhere. However, a valid total ordering is delivering m_2 then m_1 everywhere.]

(c) The following transaction satisfies the ACID properties: Alice is transferring 20 dollars from bank account B into bank account A . After five minutes, Alice checks her accounts. The amount in account A increased by 5 and the amount in account B decreased by 5. After five more minutes, Alice checks her accounts again. The amount in account A increased by 15 and the amount in account B decreased by 15.

[Answer: **False.** The isolation property is violated since there is an exposed state where part of the transaction is applied.]

(d) On the bitcoin network, if a party gathered over 51% of the computational power of the network, they could double spend.

[Answer: **True.** With a majority of the power, the party can expectedly out-mine all other processes in the network combined. This means they can spend on one chain, then begin mining on a separate fork. Eventually, this party will surpass the chain in which they spent on, regaining their coins.]

2. (20pts – 10ea) Answer the following short answer questions.

- (a) Recall the different network models, from easiest to hardest: (1) synchronous, (2) partially synchronous, (3) asynchronous.

For each of the following distributed system abstractions, write the *hardest* network model in which the abstraction can be implemented. (For example the answer for regular register with correct majority is (3).)

- deterministic uniform consensus: [Answer: (2)]
- deterministic total order broadcast: [Answer: (2)]
- atomic commit: [Answer: (1)]
- Byzantine reliable broadcast with f Byzantine processes out of $3f + 1$ total number of processes: [Answer: (3)]
- deterministic Byzantine consensus with f Byzantine processes out of $3f + 1$ total number of processes: [Answer: (2)]

- (b) Recall the equation for Nakamoto consensus:

$$\text{SHA256}(h|T|K|\text{nonce}) < D$$

First, briefly explain each component of the equation.

- SHA256 [Answer: is a hash function]
- h [Answer: is the hash of the current block/longest branch]
- T [Answer: are transactions]
- K [Answer: is miner's public key]
- none [Answer: is the integer value the miner must find to satisfy the equation]
- D [Answer: is the difficulty parameter, specifically the number of trailing zeros.]

Next, suppose we were to move the nonce outside of the SHA256 computation, like so.

$$\text{SHA256}(h|T|K) + \text{nonce} = D$$

Describe the consequences of modifying the equation in such a way. Assume D is a known constant integer.

[Answer: If we change the equation in such a manner, then the nonce is easily solvable. Specifically,

$$\text{nonce} = D - \text{SHA256}(h|T|K)$$

which is extremely fast. Since we can find the nonce quickly, then everyone can mine many blocks quickly. This is equivalent to when the difficulty is set too low. This can cause many forks in the chain.]

3. Remember the terminating reliable broadcast protocol that we saw in the class. Each process waits to either receive the message m from the sender src , or hear that $crash(src)$. Accordingly, propose either m or ϕ to consensus. When consensus decides a value at a process, it delivers the that value for TRB.

- (a) To have uniform agreement for TRB, does the consensus module need to have uniform agreement? Explain why.

[Answer: TRB with uniform agreement requires uniform consensus. Suppose we do not have a uniform consensus protocol. Then a process may decide m (and also deliver m in the TRB), then crash. Let this process be the only process that entered consensus with m . Then, let every other process will decide ϕ (and also deliver ϕ in the TRB). This violates uniform agreement. For TRB with non-uniform agreement, a non-uniform consensus is sufficient.]

- (b) Suppose we also introduce a property called *preservation*: if a correct process receives message m from src , then it eventually delivers m . Does the protocol above satisfy this property? If not, how can we modify the protocol in order to satisfy this property?

[Answer: This protocol does not satisfy preservation. Suppose src crashes and some process p_i receives $crash(src)$. Then, it will propose ϕ to the consensus module. Suppose that every other process receives m from src and proposes m to the consensus module. However, the consensus might lead to everyone deciding ϕ , and thus everyone would deliver ϕ . Therefore, preservation doesn't hold.

The modification we need is that if any process proposes m , then consensus will decide m . This is similar to atomic commit if we let m be abort, and ϕ be commit.]

4. Remember the 2-phase commit protocol for atomic commit that we saw in class. The leader requests the votes from every process. Every process responds with their vote. Based on the votes, the leader either decides 0 (abort) or 1 (commit) and broadcasts this decision. All processes decide the same upon receiving this message from the leader.

Recall the 2-phase commit protocol we saw in class for atomic commit. This protocol uses a perfect failure detector. If we know that the leader is correct, is it possible to use an eventual perfect failure detector $\diamond P$ instead?

[Answer: **No**. Even if we know the leader is correct, we cannot use an $\diamond P$ instead.

The leader requests votes, and processes respond with their votes. Suppose that $\diamond P$ tells the leader that p_i crashed. We consider two cases:

Case 1: the leader chooses 1. Then if p_i did crash, then commit-validity is violated.

Case 1: the leader chooses 0. Then if p_i did not really crash, and p_i and every other process proposed 1, then abort-validity is violated.

A correct leader can violate the properties either way.]

5. (30pts – 15ea) Byzantine fault tolerance.

- (a) Recall the Byzantine consistent broadcast protocol (that is, Byzantine reliable broadcast without the totality property) we saw in class for systems with f Byzantine processes out of $n = 3f + 1$ total number processes. A process can deliver a message m if it receives $2f + 1$ echoes of that message, as described in the pseudo-code below.

```

upon event < abeb, deliver(p, Echo(m)) > do
    echo[p] := m
    if  $\exists m : |\{p \mid \text{echo}[p]=m\}| \geq 2f + 1$  then
        trigger < deliver(s,m) >

```

Alice claims it is actually unnecessary to require $2f + 1$ echoes; rather, one can actually deliver upon receiving $n/2 + 1$ echoes. (In other words, a quorum can be size $n/2 + 1$ rather than $2f + 1$.) Her reasoning is as follows:

- i. If a process p delivers message m , then it received $n/2 + 1$ echoes for m .
- ii. If another process p' delivered a message $m' \neq m$, then p' received $n/2 + 1$ echoes for m' as well.
- iii. Each received $n/2 + 1$ echoes, meaning $n + 2$ processes sent echo messages.
- iv. However, this is impossible as there are only n processes. Thus, by contradiction, consistency is preserved.

Explain the flaw in Alice's reasoning. Use the following example to show that if a process delivers upon $n/2 + 1$ echoes, the consistency property can be violated. Let $n = 10$, $f = 3$, $\Pi = \{p_1, p_2, \dots, p_{10}\}$. Let the sender be p_1 . Let the set of Byzantine processes be $\{p_1, p_2, p_3\}$.

[Answer: The primary flaw of Alice's reasoning is that she is not accounting for the fact that Byzantine processes can echo different messages to different processes. Specifically, the senders of $n/2 + 1$ echoes to p can intersect with the senders of $n/2 + 1$ echoes to p' in up to f processes. Therefore, only $n/2 + 1 + n/2 + 1 - f = n + 2 - f$ processes are sufficient to send the messages with is less than n .

To illustrate this, we can use the example. By Alice's reasoning, a process can deliver upon receiving 6 echoes. First, we have p_1 send m to p_4, p_5, p_6 . During the echo, $p_1, p_2, p_3, p_4, p_5, p_6$ echo m to p_4 , leading it to deliver m .

Now we show that p_7 can deliver another message m' . Byzantine processes can send different echo messages to different processes. Specifically, p_1, p_2, p_3 can now echo m' to p_7 . First, we will have p_1 send m' to p_7, p_8, p_9 . During the echo step, $p_1, p_2, p_3, p_7, p_8, p_9$ will echo m' to p_7 , leading it to deliver m' .]

(b) Suppose we are operating in a Byzantine distributed system with f Byzantine processes and $n = 3f + 1$ total processes. Our system is special in that we know the first process, p_1 , is correct (and every process also knows p_1 is correct). We would like to build a Strong Byzantine Consensus (SBC) protocol using the Byzantine Reliable Broadcast (BRB/Bracha) protocol we saw in class for this system. The protocol works as follows:

- i. Process p_1 is the sender of the BRB and every other process knows this.
- ii. Process p_1 broadcasts its proposed value to the other processes.
- iii. Upon delivering from BRB, all processes decide in SBC.

For your reference, the properties of BRB are given below.

- i. **Validity:** Every message broadcast by a correct process is eventually delivered by every correct process.
- ii. **No duplication:** Every correct process delivers at most one message.
- iii. **Integrity:** If a correct process delivers m with sender p , and p is correct, then p has broadcast m .
- iv. **Consistency:** If a correct process delivers m and another correct process delivers m' , then $m = m'$.
- v. **Totality:** If some correct process delivers a message, then every correct process eventually delivers a message.

Using the properties of BRB, justify that the above protocol satisfies the properties of SBC. In the proof of each of the consensus properties below, underline which property/properties of BRB are used.

Termination (every correct process eventually decides some value):

[Answer: **Validity.** Since p_1 is correct, then every correct process will eventually deliver its message. Since a process decides upon delivery, then it will also decide eventually.]

Strong Validity (A correct process can only decide a value that was proposed by a correct process or special value $\square$. Further, if all correct processes propose the same value v , then no correct process decides a value different from v):

[Answer: The protocol clearly has every correct process decide the proposed value of p_1 , and p_1 is known to be correct. Therefore a correct process can only ever decide a value proposed by a correct process. Additionally, if all correct processes propose v , then of course p_1 also proposes v . Then, no correct process would decide something different than what p_1 proposed.]

Integrity (No correct process decides twice):

[Answer: **No Duplication.** Since a process decides upon delivery, and every process delivers at most one message, then it can decide at most once.]

Agreement (No two correct processes decide differently):

[Answer: **Consistency.** Assume that two correct processes decide differently. Since processes only decide upon delivery of a message, it implies that the two processes delivered different messages. However, by consistency, no two correct processes delivery different messages.]